Intractability

CSE 211 (Theory of Computation)

Atif Hasan Rahman

Assistant Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering & Technology

Adapted from slides by

Dr. Muhammad Masroor Ali

Intractability

- We now turn to efficient versus inefficient computation
- We focus on problems that are decidable, and ask which of them can be computed by Turing machines that run in an amount of time that is polynomial in the size of the input
  - The problems solvable in polynomial time on a typical computer are exactly the same as the problems solvable in polynomial time on a Turing machine
  - Practical problems requiring polynomial time are usually solvable in an amount of time that we can tolerate, while those that require exponential time generally cannot be solved except for small instance

The Class $\mathcal{P}$

- A Turing machine M is said to be of time complexity $T(n)$ or to have running time $T(n)$, if whenever M is given an input w of length n, M halts after making at most $T(n)$ moves
- A language L is in class $\mathcal{P}$ if there is some polynomial $T(n)$ such that $L = L(M)$ for some deterministic TM M of time complexity $T(n)$
- $\mathcal{P}$ is the class of problems solvable in polynomial time by deterministic TM (or computer)

Examples of Problems in $\mathcal{P}$

- Minimum spanning tree
- Shortest path
- Edit distance
- Maximum flow

The Class $\mathcal{NP}$

- A language L is in the class $\mathcal{NP}$ (nondeterministic polynomial) if there is a *nondeterministic* TM M and a polynomial time complexity $T(n)$ such that $L = L(M)$ and when M is given an input of length n, there are no sequences of more than $T(n)$ moves of M
- $\mathcal{NP}$ is the class of problems checkable in polynomial time

Examples of Problems in $\mathcal{NP}$

- Everything in $\mathcal{P}$
- Satisfiability
- Hamiltonian path
- Traveling salesman problem
- Vertex cover
- Independent set

Relation between $\mathcal{P}$ and $\mathcal{NP}$

- Since every deterministic TM is a nondeterministic TM that happens never to have a choice of moves

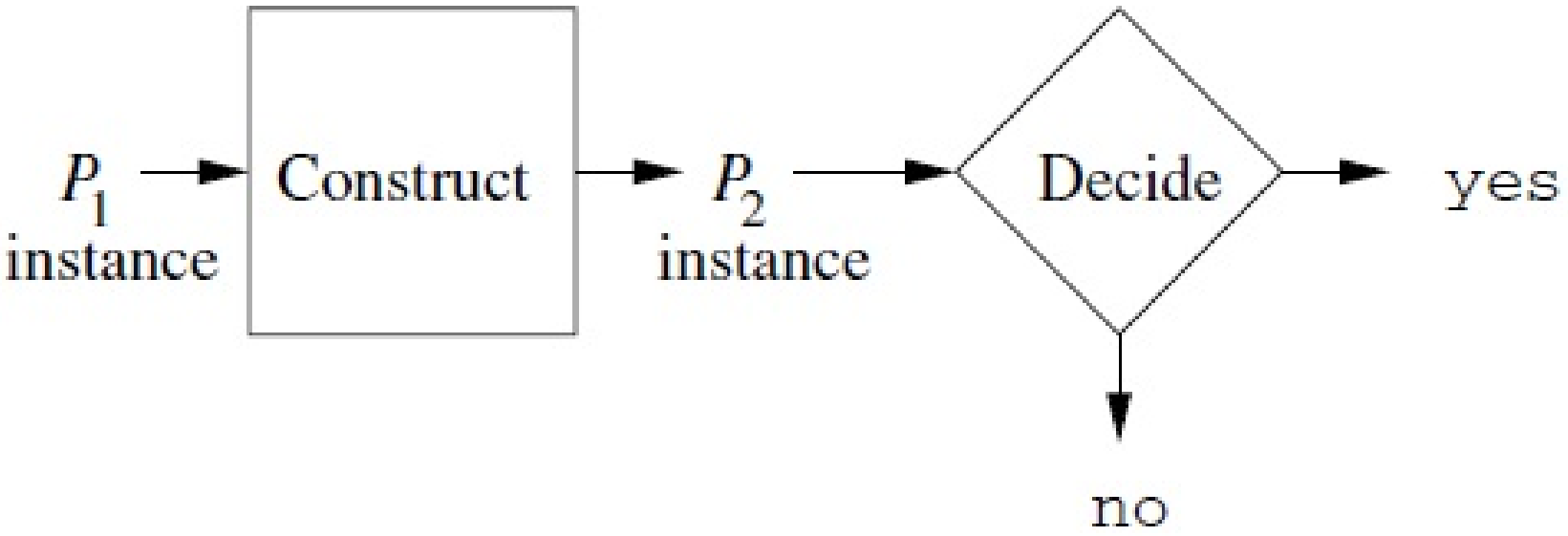
$$\mathcal{P} \subseteq \mathcal{NP}$$

- However, it appears that $\mathcal{NP}$ contains many problems not in $\mathcal{P}$
  - The intuitive reason is that a NTM running in polynomial time has the ability to guess an exponential number of possible solutions to a problem and check each one in polynomial time in parallel
- One of the deepest open questions of Mathematics whether $\mathcal{P}$=$\mathcal{NP}$

A note

- The classes $\mathcal{P}$ and $\mathcal{NP}$ are technically defined for decision problems i.e. ones with yes/no answer
- We can convert an optimization problem by adding a parameter k and asking whether there is a solution with value at most/at least k
  - If this can be answered in polynomial time, we can find optimal solution by doing binary search over range of possible solution values in polynomial time

Polynomial-Time Reductions



- Given an instance of P_1, we construct an instance of P_2 in polynomial time, such that if instance of P_2 can be decided, we can also decide the instance of P_1

NP-Complete Problems

- L is NP-complete if the following statements are true:
  1. L is in $\mathcal{NP}$
  2. For every language L' in $\mathcal{NP}$, there is a polynomial-time reduction of L' to L
- **If** a single NP-complete problem can be solved in polynomial time, then every problem in $\mathcal{NP}$ can be solved in polynomial time, implying $\mathcal{P} = \mathcal{NP}$

Easy vs Hard Problems

Hard problems (**NP**-complete)	Easy problems (in **P**)
3SAT	2SAT, HORN SAT
TRAVELING SALESMAN PROBLEM	MINIMUM SPANNING TREE
LONGEST PATH	SHORTEST PATH
3D MATCHING	BIPARTITE MATCHING
KNAPSACK	UNARY KNAPSACK
INDEPENDENT SET	INDEPENDENT SET on trees
INTEGER LINEAR PROGRAMMING	LINEAR PROGRAMMING
RUDRATA PATH	EULER PATH
BALANCED CUT	MINIMUM CUT

NP-Complete Problems

Theorem

If P_1 is NP-complete, P_2 is in NP, and there is a polynomial time reduction of P_1 to P_2 then P_2 is NP-complete

PROOF: We need to show that every problem L in $\mathcal{NP}$ polynomial-time reduces to P_2

- Since P_1 is NP-complete, we know that there is a polynomial time reduction of L to P_1
- If there is a polynomial time reduction of P_1 to P_2, we can combine this with the above reduction to get a polynomial time reduction from L to P_2
- Since L could be any language in $\mathcal{NP}$, we have shown that all of $\mathcal{NP}$ polynomial time reduces to P_2 i.e., P_2 is NP-complete

NP-Complete Problems

Theorem

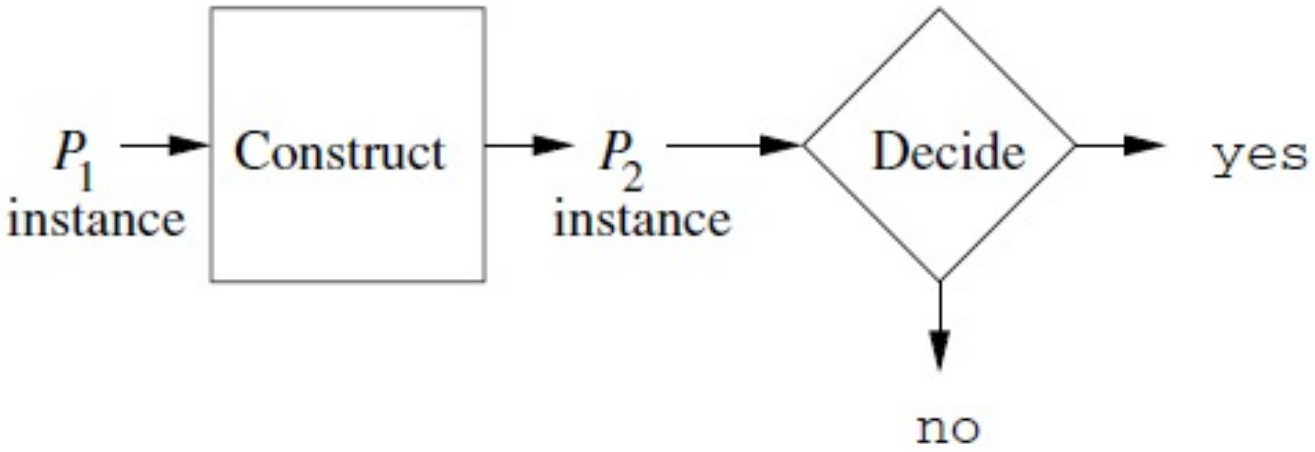
If some NP-complete problem P is in $\mathcal{P}$ then $\mathcal{P}$=$\mathcal{NP}$

PROOF: Suppose P is both NP-complete and in $\mathcal{P}$. Then all languages L in $\mathcal{NP}$ reduce in polynomial time to P. If P is in $\mathcal{P}$, then L is in $\mathcal{P}$. So, $\mathcal{P}$=$\mathcal{NP}$

Proving a Problem to be NP-Complete

To prove that a problem P_2 is NP-complete, we need to

- Show P_2 is in $\mathcal{NP}$
- Find an NP-complete problem P_1 which reduces to P_2 in polynomial time. This shows that if someone finds a polynomial algorithm for P_2, then $\mathcal{P}$=$\mathcal{NP}$

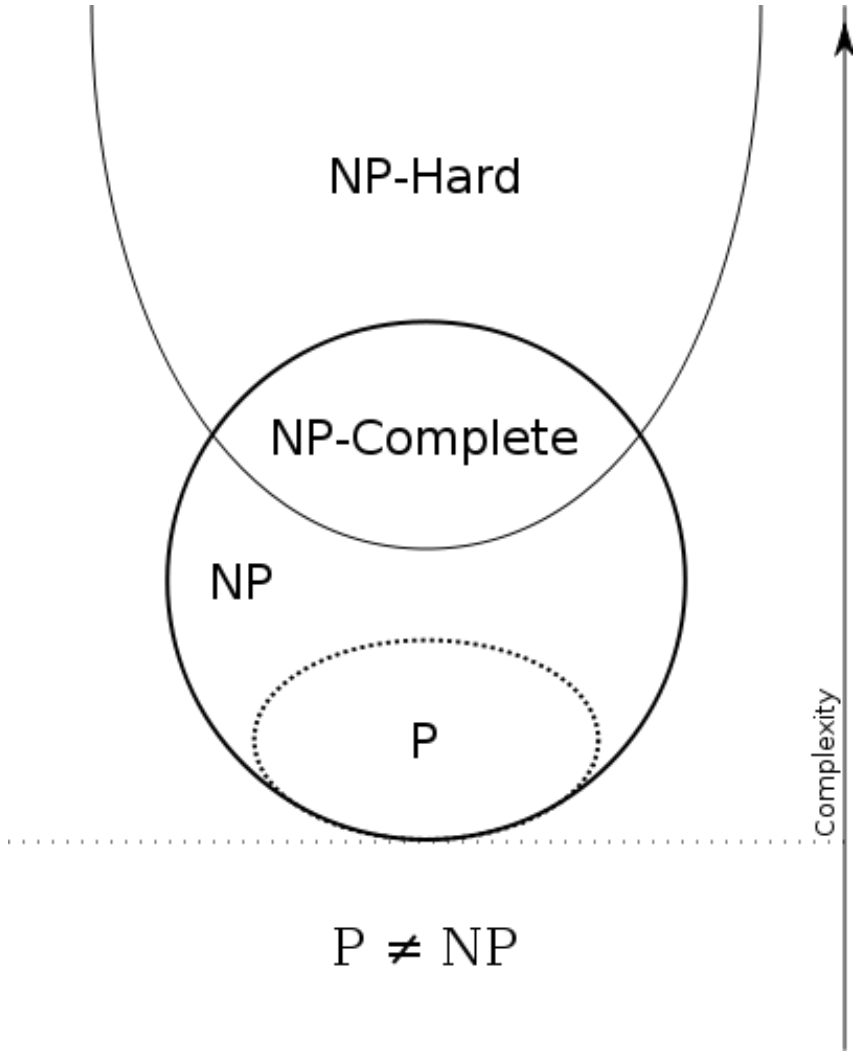


Note the direction of reduction

NP-Hard Problems

- Some problems L are so hard that
  - although we can prove condition (2) of the definition of NP-completeness (every language in NP reduces to L in polynomial time)
  - we cannot prove condition (1) that L is in $\mathcal{NP}$
- If so, we call L ***NP-hard***.

Likely Relation among Problem Classes



A first NP-complete problem

- To prove NP-completeness, we usually show a poly time reduction from some NP-complete problem
- NP-complete problems are thus linked by chains of polynomial time reductions
- And we claimed that if one NP-complete can be solved in polynomial time, all problems in NP can be solved in polynomial time
- But for this we need a first problem in NP to which every problem in NP can be reduced to
- Satisfiability is our first NP-complete problem

Satisfiability - a first NP-complete problem

- A Boolean formula is an expression involving
  - Boolean variables
  - Logical operators - AND ($\wedge$), OR ($\vee$) and NOT ($\neg$)
  - Parentheses to group operators and operands

$$((x1 \wedge x2) \vee (x3 \wedge \neg x4)) \wedge (\neg x1 \vee \neg x2) \wedge (x2 \vee \neg x3)$$

- A truth assignment is assignment of either true or false to each of the variables in the expression
- A truth assignment satisfies a Boolean expression if the truth assignment makes the expression true
- A Boolean expression is said to be satisfiable if there exists at least one truth assignment that satisfies it

Satisfiability - a first NP-complete problem

Satisfiability (SAT)

The satisfiability problem (SAT) is, given a Boolean expression, to decide is it satisfiable.

NP-completeness of the SAT problem

Theorem (Cook's Theorem)

SAT is NP-complete

PROOF: The first part of the proof is showing that SAT is in $\mathcal{NP}$. This part is easy and we'll skip.

NP-completeness of the SAT problem

- Now, we must prove that if L is any language in NP, then there is a polynomial time reduction of L to SAT
- We may assume that there is some single-tape NTM M and a polynomial $p(n)$ such that M takes no more than $p(n)$ steps on an input of length n along any branch
- Thus, if M accepts an input w, and $|w| = n$, then there is a sequence of moves of M such that
  - α_0 is the initial ID of M with input w
  - $\alpha_0 \vdash \alpha_1 \vdash \cdots \vdash \alpha_k$, where $k \leq p(n)$
  - α_k is an ID with an accepting state

NP-completeness of the SAT problem

- The main idea behind the reduction is to map NTM M and arbitrary Turing machine input w to a Boolean formula $F(M, w)$ whose satisfiability is equivalent to the existence of some computation branch of $T(M, w)$ that is accepting.
- The clever idea in the proof is that the reduction algorithm does not bother to understand the steps of each of the branches of $T(M, w)$
- Instead it produces Boolean formulas that allow for the simulation of any branch of $T(M, w)$ via an appropriate truth assignment
- Thus the "complexity and structure" of $T(M, w)$ remains after the reduction, and that complexity is witnessed by the exponentially large number of possible truth assignments that yield the different simulations of the different branches

NP-completeness of the SAT problem

The variables of $F(M, w)$ to keep track of the computation along any one branch of the tree $T(M, w)$.

1. **Current state of the machine.** For each $q \in Q$ and for each $0 \leq j \leq p(n)$, $s(q, j)$ is the Boolean variable whose truth is equivalent to the statment that "M is in state q during the j th step of the computation branch". Which computation branch? One that hopefully ends in accepting!
2. **Location of the tape head.** For each $0 \leq i, j \leq p(n)$, l_{ij} is true iff the tape head is located at cell i during step j of the computation.
3. **Contents of each cell.** For each $t \in \Gamma$ and for each $0 \leq i, j \leq p(n)$, $c(i, j, t)$ is true iff cell i contains symbol t during step j of the computation.
4. **Unchanging of a cell.** for every $0 \leq i \leq p(n)$ and for every $0 \leq j \leq p(n) - 1$, $u(i, j)$ is true iff cell i is unchanged from step j to step $j + 1$ of the computation.

NP-completeness of the SAT problem

Rule 1. A Turing machine is in exactly one state during any step of the computation. For every $0 \leq j \leq p(n)$,

$$\bigvee_{q \in Q} s(q, j),$$

and for every $q \in Q$,

$$s(q, j) \rightarrow \bigwedge_{\hat{q} \in Q - \{q\}} \overline{s(\hat{q}, j)}.$$

Rule 2. A Turing machine tape head is in exactly one location during any step of the computation. For every $0 \leq j \leq p(n)$,

$$\bigvee_{i=0}^{p(n)} l_{ij},$$

and for every $0 \leq k \leq p(n)$,

$$l_{kj} \rightarrow \bigwedge_{i \neq k} \overline{l_{ij}}.$$

NP-completeness of the SAT problem

Rule 3. Each tape cell of Turing machine M contains exactly one symbol from Γ during any step of the computation. For every $0 \leq i, j \leq p(n)$,

$$\bigvee_{t \in \Gamma} c(i, j, t),$$

and for every $t \in \Gamma$,

$$c(i, j, t) \rightarrow \bigwedge_{\hat{t} \in \Gamma - \{t\}} \overline{c(i, j, \hat{t})}.$$

Rule 4. If a tape cell remains unchanged from one step to the next, then the cell must contain the same symbol from one step to the next. For every $0 \leq i \leq p(n)$, for every $0 \leq j \leq p(n) - 1$, and for every $t \in \Gamma$,

$$c(i, j, t) \wedge u(i, j) \rightarrow c(i, j + 1, t).$$

NP-completeness of the SAT problem

Rule 5. Given a Turing machine configuration, then next configuration is determined by application of the δ transition function. For every $0 \leq i \leq p(n)$, for every $0 \leq j \leq p(n) - 1$, for every $t \in \Gamma$, and for every $q \in Q$, let

$$\delta(q, t) = \{(q_1, t_1, d_1), \ldots, (q_m, t_m, d_m)\}.$$

Then

$$[s(q, j) \wedge l_{ij} \wedge c(i, j, t)] \rightarrow$$

$$\bigwedge_{k \neq i} u(k, j) \wedge [(s(q_1, j+1) \wedge l_{\phi(1)(j+1)} \wedge c(i, j+1, t_1)) \vee \cdots$$

$$\vee (s(q_m, j+1) \wedge l_{\phi(m)(j+1)} \wedge c(i, j+1, t_m))],$$

where, for $1 \leq k \leq m$,

$$\phi(k) = \begin{cases} i-1 & \text{if } d_k = \text{L and } i > 0 \\ i & \text{if } d_k = \text{L and } i = 0 \text{ or } d_k = \text{R and } i = p(n) \\ i+1 & \text{if } d_k = \text{R and } i < p(n) \end{cases}$$

NP-completeness of the SAT problem

Rule 6. The initial configuration of the computation of M on input w begins in state q_0, with the tape head located at cell 0, and with the first $p(n)+1$ tape cells reading $w(B)^{p(n)+1-n}$. (w followed by $p(n)+1-n$ blanks)
This family has just one formula!

$$s(q_0,0) \wedge l_{00} \wedge c(0,0,w_1) \wedge \cdots \wedge c(n-1,0,w_n) \wedge c(n,0,B) \wedge \cdots$$

$$\wedge c(p(n),0,B).$$

Rule 7. The final configuration of a computation branch of $T(M,w)$ is accepting. Recall that we want $F(M,w)$ to be satisfiable if and only if some computation branch of $T(M,w)$ is accepting. This family also has just one formula, but it is pivotal for ensuring that satisifiability will be equivalent with acceptance. That formula is simply $s(q_a,p(n))$.

NP-completeness of the SAT problem

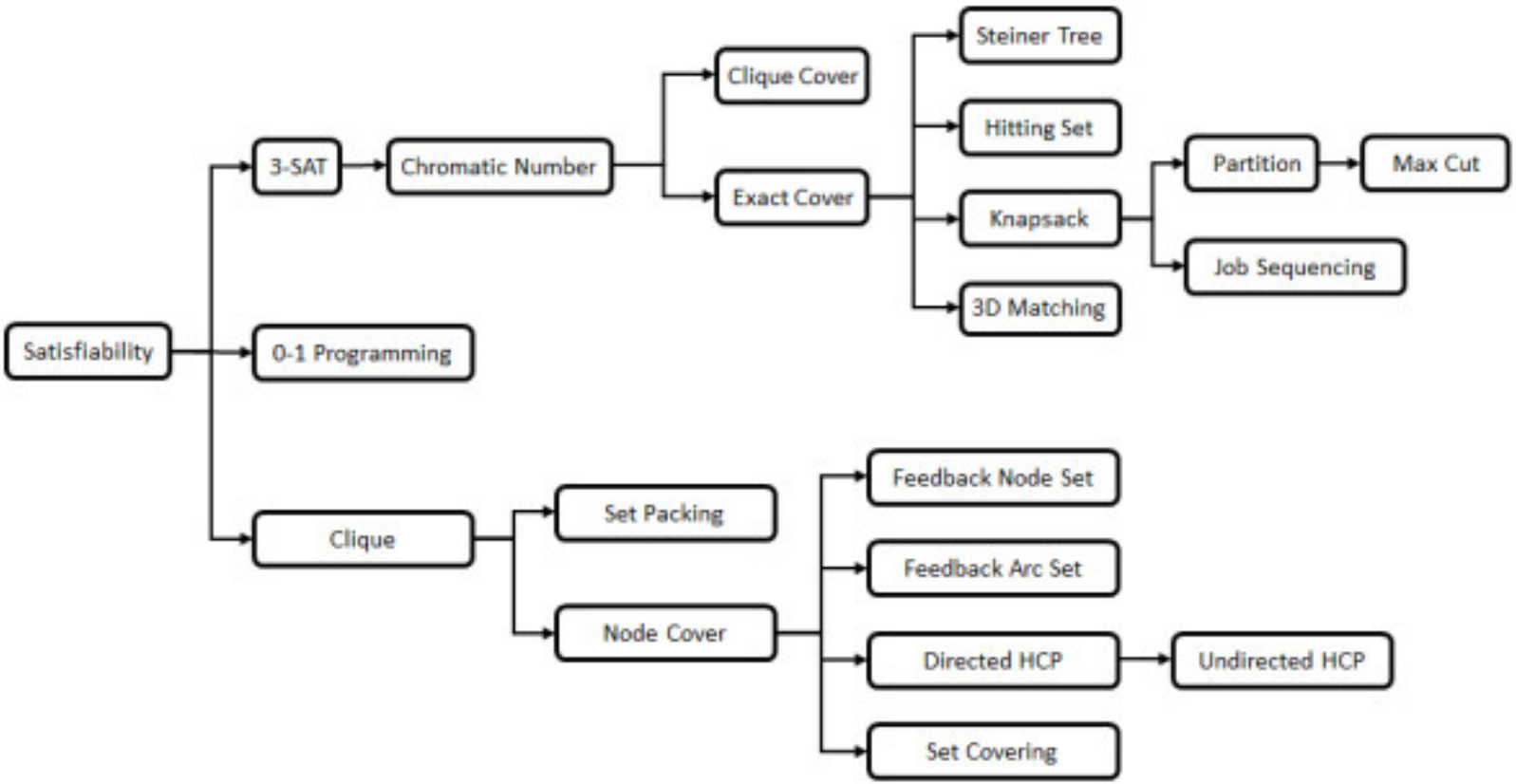
Lemma

Let $F(M, w)$ be the AND of all formula in each of the above seven Boolean formula families. Then $F(M, w)$ is effectively computable given M and w, and $|F(M, w)|$ is bounded by a polynomial in the length n of w.

- The effective computability of $F(M, w)$ follows from the fact that each Boolean formula from each family can be written effectively given each of the parameters that it depends upon
- A study of each formula family and the number of formulas in each family finds that Rule 5 yields the asymptotically fastest growing family at $O(|Q||\Gamma|(p(n))^3)$ which is a polynomial

Karp's 21 NP-complete problems

- In 1972, Richard Karp showed 21 problems to be NP-complete



Graph Coloring

Graph Coloring

The coloring problem is, given a graph G and an integer k, is G k-colorable, that is, can we assign one of k colors to each node of G in such a way that no edge has both of its ends colored with the same color

3-coloring

The 3-coloring problem is, given a graph G, is it 3-colorable?

NP-completeness of Graph Coloring

Theorem

3-Coloring is NP-complete

PROOF: The first part of the proof is showing that 3-Coloring is in $\mathcal{NP}$. We just check whether two adjacent vertices are assigned the same color.

NP-completeness of 3-Coloring

We will reduce the 3-SAT problem to 3-coloring

- We define nodes v_i and $\overline{v_i}$ corresponding to each variable x_i and its negation $\overline{x_i}$.
- We also define three "special nodes" T, F, and B, which we refer to as True, False, and Base.

NP-completeness of 3-Coloring

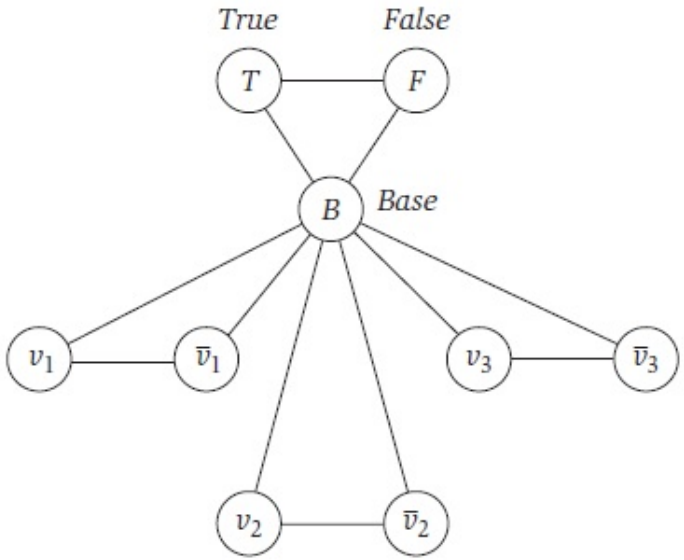


Figure 8.11 The beginning of the reduction for 3-Coloring.

- In any 3-coloring of G, the nodes v_i and $\overline{v_i}$ must get different colors, and both must be different from Base.
- In any 3-coloring of G, the nodes True, False, and Base must get all three colors in some permutation. This means that for each i, one of v_i and $\overline{v_i}$ gets the True color, and the other gets the False color

NP-completeness of 3-Coloring

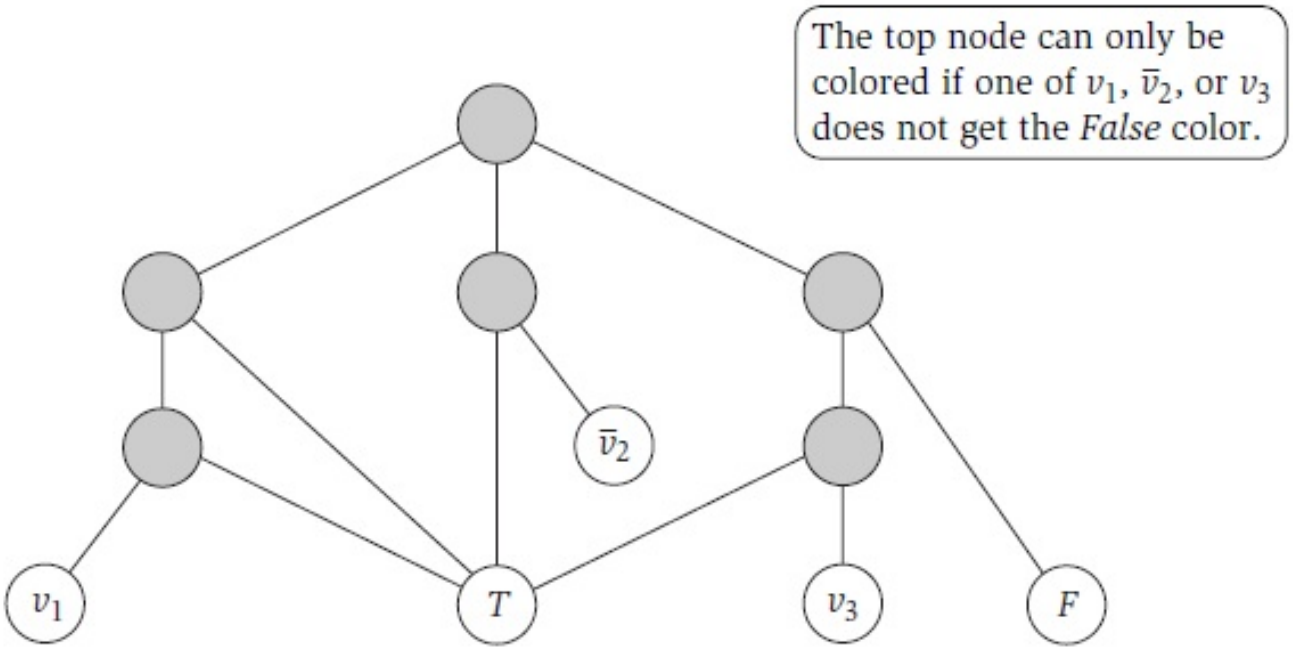
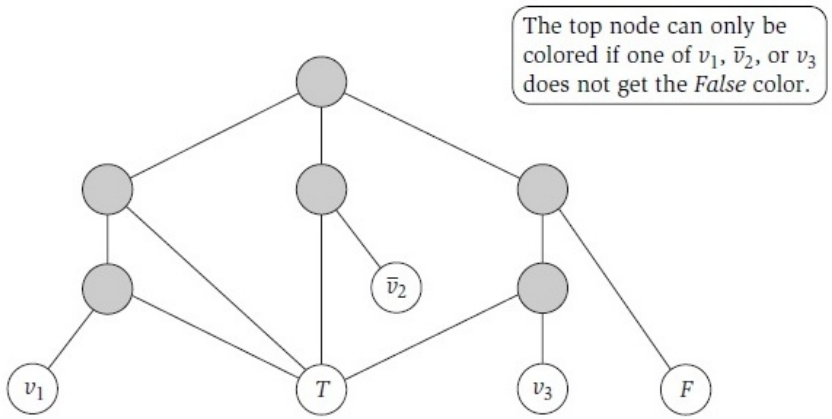


Figure 8.12 Attaching a subgraph to represent the clause $x_1 \vee \overline{x}_2 \vee x_3$.

- For each clause in the 3-SAT instance, we attach a six-node subgraph as shown

NP-completeness of 3-Coloring



- Now suppose that in some 3- coloring of G all three of v_1, $\overline{v_2}$, and v_3 are assigned the False color.
- Then the lowest two shaded nodes in the subgraph must receive the Base color, the three shaded nodes above them must receive, respectively, False, Base, and True
- Hence there's no color that can be assigned to the topmost shaded node.

NP-completeness of 3-Coloring

- Suppose that there is a satisfying assignment for the 3-SAT instance.
- We define a coloring of G' by first coloring Base, True, and False arbitrarily with the three colors, then, for each i, assigning v_i the True color if $x_i = 1$ and the False color if $x_i = 0$.
- $\overline{v_i}$ is assigned the only available color
- It is thus possible to extend this 3-coloring into each six-node clause subgraph, resulting in a 3-coloring of all of G'

NP-completeness of 3-Coloring

- Conversely, suppose G' has a 3-coloring. In this coloring, each node v_i is assigned either the True color or the False color; we set the variable x_i correspondingly.
- In each clause of the 3-SAT instance, at least one of the terms in the clause has the truth value 1
- For if not, then all three of the corresponding nodes has the False color in the 3-coloring of G' and, as we have seen above, there is no 3-coloring of the corresponding clause subgraph consistent with this - a contradiction.

Maximum Common Subgraph

Maximum Common Subgraph

In the maximum common subgraph problem, we are given two graphs $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$, and an integer k. The problem is to decide whether G_1 and G_2 have a common induced subgraph with at least k vertices.

Maximum Clique Problem

Maximum Clique Problem

Given a graph $G = (V, E)$ and an integer k, the problem is to decide whether G has a set of mutually adjacent vertices of size at least k vertices.

NP-completeness of Clique

Theorem

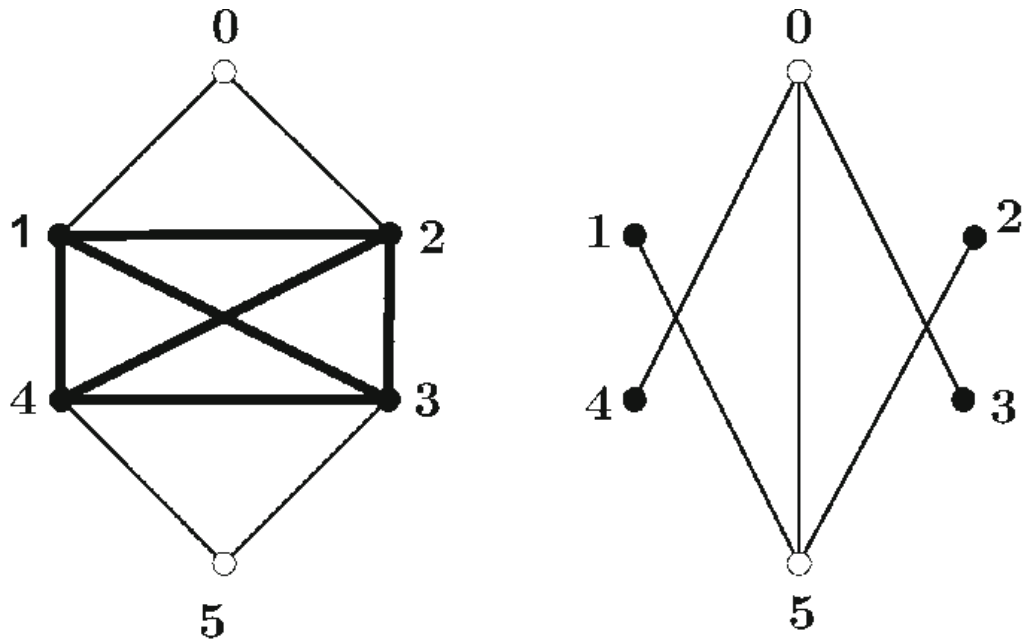
Maximum Clique is NP-complete

- Clique is in $\mathcal{NP}$
- Given an instance of the independent set problem, $G = (V, E)$ and k, construct an instance of clique $G' = (V', E')$ and k' as follows:

$$V' = V, E' = \{(u, v) | (u, v) \notin E\}, k' = k$$

- G' has a clique of size $\geq k'$ iff G has an independent set of size $\geq k$

NP-completeness of Clique



NP-completeness of Maximum Common Subgraph

Theorem

Maximum Common Subgraph is NP-complete

- Maximum Common Subgraph is in $\mathcal{NP}$
- Reduction from clique

NP-completeness of Maximum Common Subgraph

- Given an instance of clique, $G = (V, E)$ and k, construct an instance of maximum common subgraph $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$, and k' as follows:
  - $G_1 = G$
  - $G_2 =$ complete graph with |V| vertices
  - $k' = k$
- Since G_2 is a complete graph, any subgraph must also be a complete graph, and thus a clique. So if there is a common subgraph of G_1 and G_2 of size at least k, then this must be a clique of size at least k
- Conversely, suppose that there is a clique V' of size k in G. Then clearly it forms a common subgraph of size k with any subgraph of G_2 of size k